Bản báo cáo tiến độ lần 2

I. Chủ đề nhóm (Nhóm 9)

TRACE CACHE

II. Thành viên nhóm

Họ và tên	Giới tính	Lớp	MSSV
Trần Anh Khoa (NT)	Nam	64.CNTT-2	64131011
Nguyễn Trọng Thời	Nam	64.CNTT-2	64132333
Trần Duy Khánh	Nam	64.CNTT-2	64130994
Nguyễn Minh Hùng	Nam	64.CNTT-2	64130776
Lê Trọng Tài	Nam	64.CNTT-2	64132074

III. Chủ đề tìm hiểu lần 2

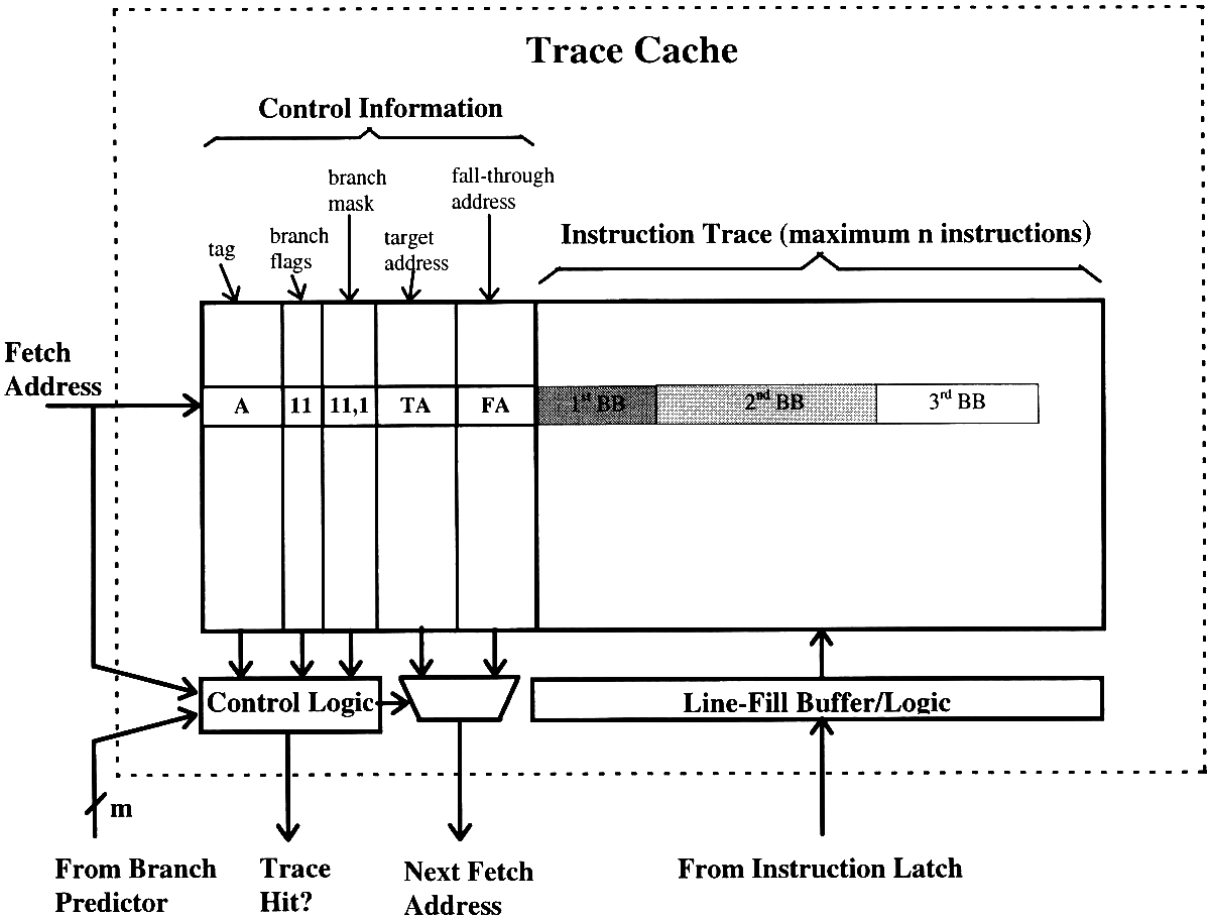
Tuần	Phân chia việc làm	Nội dung cụ thể	Đánh giá
8,9	- Tìm hiểu nội dung phần 3, tìm kiếm các sơ đồ liên quan	- Cả nhóm hợp tác tìm kiếm nội dung cùng nhau	- Mọi người trong nhóm tích cực tìm kiếm tài liệu, nội dung. - Tuy nhiên vẫn còn 1 số bạn nộp bài không chỉnh chu. - Bản word chỉ là nội dung được copy từ 1 bài báo
10,11	- Nêu ra và tóm tắt được nội dung phần 4 đồng thời phải phân tích làm rõ nó	- Tài, Hùng, Khánh sẽ tìm ý và viết thành đoạn văn - Thời, Khoa sẽ phân tích cơ chế của Trace Cache	
12	- Tìm hiểu về nội dung phần 5	- Cả nhóm cùng tìm kiếm cùng 1 nội	

		dung, rồi từ đó sẽ bổ sung ý cho nhau cũng như là thể hiện nội dung tốt hơn.	nào đó và được dịch thuật bằng công cụ, không trâu chuốt từ ngữ, thậm chí còn chưa gỡ hyperlink. - Đối với lần này thì bạn Hùng chưa nộp nội dung cho phần 3 và phần 5. - Bản đúc kết cụ thể nội dung của lần này sẽ được gửi vào tiến độ lần sau luôn ạ.

IV. Nội dung chi tiết

❖Trần Anh Khoa

NỘI DUNG 3:



Hình 1. SƠ ĐỒ CẤU TẠO CỦA CACHE

Cấu trúc của 1 trace cache bao gồm 2 phần: thông tin điều khiển (Control Information) và Vết chỉ dẫn (Instruction Trace).

Phần Control Information lại bao gồm Control Logic và địa chỉ của ô nhớ sẽ được tìm nạp tiếp theo hay còn gọi là Next Fetch Address.

Trong phần Control Logic, Trace Cache sẽ nhận địa chỉ tìm nạp và thông qua tag, branch flags, branch mask để biết được trace hit hay là trace miss. Nếu là trace miss thì trace cache này phải đợi. Còn là trace hit thì sẽ chuyển đến Fetch Address tiếp theo.

Điểm qua về khái niệm của 1 số thuật ngữ sử dụng trong Trace Cache:

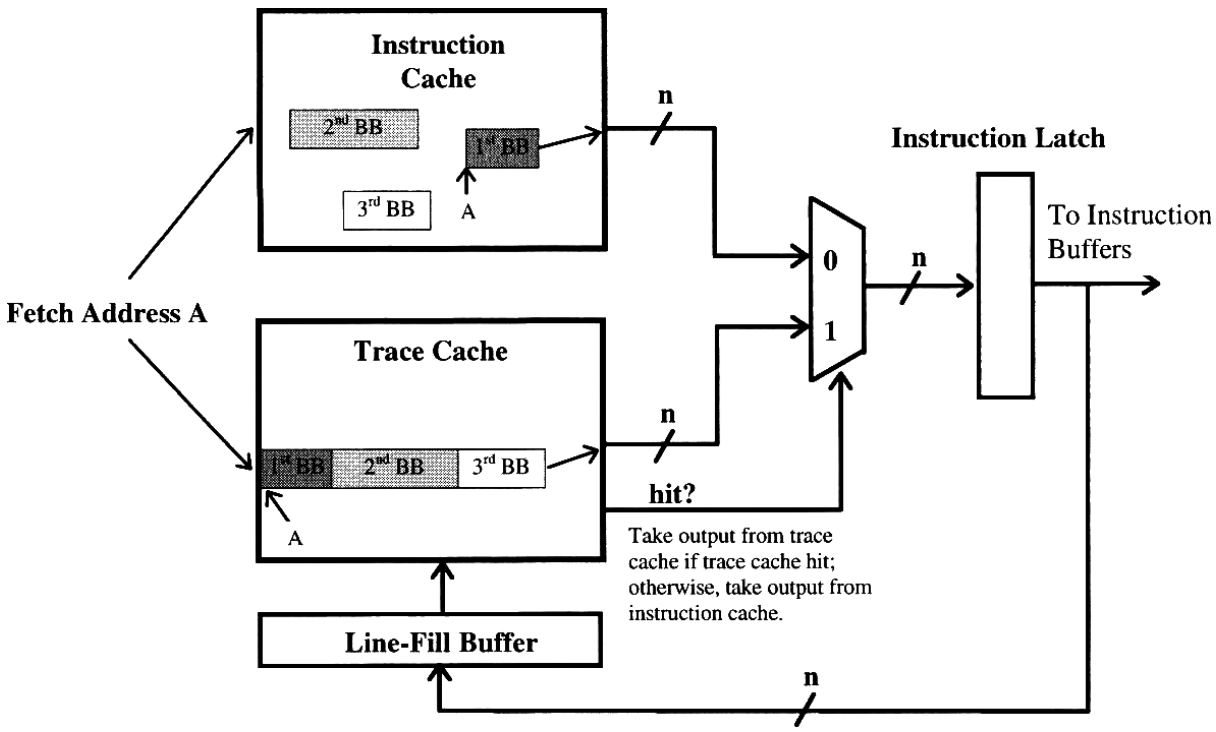
- Tag ở đây có tác dụng xác định địa chỉ bắt đầu của vết, tức là các bit bậc cao hơn của địa chỉ cho chỉ dẫn đầu tiên của vết.

- Branch Flags là một bit duy nhất cho mỗi nhánh trong vết để cho biết có hay không nhánh đã được thực hiện hoặc không được thực hiện.
- Branch Mask: Cần có một số trạng thái để chỉ ra số lượng nhánh trong dấu vết và điều này cần Branch Mask quyết định. Thông tin này được cả bộ đệm theo dõi sử dụng để xác định có bao nhiêu cờ nhánh cần kiểm tra và bởi bộ dự đoán nhánh để cập nhật.
- Đến với phần Fetch Address tiếp theo thì ta có Target Address và Fall-through Address.
- Target Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng là một nhánh và đượxc dự đoán là đã thực hiện.
- Fall-through Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng của vết là một nhánh và được dự đoán là không được thực hiện.

Instruction cache được tạo ra bởi Line-Fill Buffer / Logic hay còn gọi là bộ đệm điền dòng hoặc logic điền dòng.

Bộ đệm điền dòng được sử dụng để chốt các khối cơ bản lại với nhau để tạo thành các vết được ghi vào trace cache. Các khối cơ bản được chốt từng cái một vào bộ đệm điền khi tiến hành tìm nạp lệnh. Logic điền (Fill logic) sẽ hợp nhất từng khối cơ bản mới với các hướng dẫn đã có trong bộ đệm điền vào dòng. Quá trình điền hoàn tất khi đạt được độ dài vết tối đa n hoặc số lượng nhánh tối đa trong vết m.

NỘI DUNG 4:



Hình 1. CƠ CHẾ TÌM NẠP

Sơ đồ tìm nạp sử dụng một trace cache hoạt động cùng với Standard instruction cache (Bộ đệm chỉ dẫn tiêu chuẩn). Vào đầu mỗi chu kỳ, Trace cache được truy cập song song với trace instruction dựa trên địa chỉ tìm nạp. Công cụ dự đoán nhiều nhánh (multiple branch predictor) tạo ra các kết quả nhánh cho một vài nhánh tiếp theo trong khi các bộ đệm đang được truy cập. Nếu có một lần truy cập được vào trace cache (trace cache hit), thì vết sẽ được đọc từ trace cache vào bộ đệm phát hành lệnh (Instruction issue buffer). Một trace cache hit xảy ra khi địa chỉ tìm nạp khớp với thẻ địa chỉ của vết và dự đoán nhánh khớp với hướng nhánh của dấu vết. Nếu một trong hai điều kiện này không đúng, thì sẽ xảy ra lỗi trace cache (trace cache miss) và quá trình tìm nạp lệnh thông thường được tiến hành từ instruction cache.

NỘI DUNG 5:

Logic hit và miss của trace cache cũng tương tự như trên cache. Tức là các trace line được lưu trữ trong trace cache dựa trên PC của lệnh đầu tiên trong vết và một tập hợp các dự đoán rẽ nhánh. Điều này cho phép lưu trữ các trace line khác nhau bắt đầu trên cùng một địa chỉ, mỗi line đại diện cho các

kết quả nhánh khác nhau. Phương pháp gắn thẻ này giúp cung cấp khả năng kết hợp đường dẫn đến bộ nhớ cache theo dõi. Phương pháp khác có thể bao gồm việc chỉ khởi động PC dưới dạng thẻ trong trace cache. Trong giai đoạn tìm nạp lệnh của một quy trình, PC hiện tại cùng với một tập hợp các dự đoán nhánh được kiểm tra trong bộ nhớ trace cache để tìm lần truy cập (hit). Nếu có một lần hit, một trace line sẽ được cung cấp cho đơn vị tìm nạp mà không cần phải chuyển đến bộ đệm thông thường hoặc bộ nhớ cho các chỉ dẫn này. Trace cache tiếp tục cung cấp nguồn cho đơn vị tìm nạp cho đến khi trace line kết thúc hoặc cho đến khi có dự đoán sai trong đường dẫn. Nếu có sai sót (miss), một dấu vết mới sẽ bắt đầu được xây dựng.

❖Nguyễn Trọng Thời

NỘI DUNG 3:

Đây một sơ đồ khối cho Trace Cache, một cấu trúc bộ nhớ đệm đặc biệt được sử dụng trong kiến trúc máy tính để cải thiện hiệu suất trong việc phân tích, dự đoán và thực thi các lệnh. Sau đây là các thuật ngữ được đề cập trong sơ đồ:

- Trace Cache: Cấu trúc bộ nhớ đệm đặc biệt để lưu trữ các đường dẫn thực thi (trace) của các chương trình đã được phân tích trước đó.
- Control Information: Các thông tin điều khiển liên quan đến các trace, bao gồm cả branch mask, fall-through address, tag và branch flags.
- Branch Mask: Một biểu diễn nhị phân của các câu lệnh nhảy (branch) trong trace, dùng để xác định các câu lệnh nhảy có nên được thêm vào trace hay không.
- Fall-through Address: Địa chỉ thực thi tiếp theo của các câu lệnh sau một câu lệnh không nhảy trong trace.

- Tag: Một giá trị đánh dấu để định danh các trace khác nhau.
- Branch Flags: Một tập hợp các cờ báo hiệu các câu lệnh nhảy trong trace, để kiểm soát dòng thực thi.
- Target Address: Địa chỉ đích của một câu lệnh nhảy trong trace.
- Fetch Address (FA): Địa chỉ bắt đầu của trace trong bộ nhớ lệnh.
- A, 11, 11,1, TA: Các tín hiệu điều khiển khác trong trace cache.
- Instruction Trace (maximum n instructions): Số lượng tối đa các lệnh trong một trace.
- 1 BB, 2nd BB, 3d BB: Các trace cache hit và tất cả các basic block (BB) được lấy từ trace cache.
- Control Logic: Một cấu trúc điều khiển để quản lý trace cache và tạo các trace mới.
- Line-Fill Buffer/Logic: Một bộ đệm dùng để lưu trữ các dòng dữ liệu trong trace cache.
- m: Kích thước của trace cache.
- From Branch Predictor: Dự đoán điều kiện nhảy sẽ xảy ra trong chương trình.
- Trace Hit?: Câu hỏi liên quan đến việc trace hiện tại có được tìm thấy trong trace cache hay không.
- Next Fetch Address: Địa chỉ bắt đầu của trace tiếp theo trong bộ nhớ lệnh.
- From Instruction Latch: Các tín hiệu điều khiển được lấy từ các lệnh trong trace.

Sơ đồ khối Trace Cache là một cấu trúc bộ nhớ đệm đặc biệt được sử dụng trong kiến trúc máy tính để cải thiện hiệu suất trong việc phân tích, dự đoán và thực thi các lệnh. Đây là một số bước hoạt động và cơ chế của Trace Cache:

- Fetch Address (FA): Trace Cache nhận địa chỉ bắt đầu của trace từ bộ nhớ lệnh.
- Control Logic: Control Logic kiểm tra xem trace hiện tại có được lưu trữ trong Trace Cache hay không. Nếu trace được tìm thấy trong Trace Cache, nó được gọi là Trace Hit và Control Logic chuyển đổi đến các lệnh trong trace. Nếu Trace Cache không có trace hiện tại, Control Logic sẽ tạo một trace mới bằng cách lấy các lệnh từ bộ nhớ lệnh và thêm chúng vào Trace Cache.
- Instruction Trace: Trace Cache lưu trữ một số lượng tối đa các lệnh trong một trace.
- Control Information: Trace Cache lưu trữ các thông tin điều khiển liên quan đến các trace, bao gồm cả branch mask, fall-through address, tag và branch flags.
- Line-Fill Buffer/Logic: Một bộ đệm dùng để lưu trữ các dòng dữ liệu trong Trace Cache. Khi Trace Cache lấy dữ liệu từ bộ nhớ lệnh, nó được lưu trữ trong Line-Fill Buffer/Logic.
- Basic Block: Trace Cache lưu trữ các Basic Block (BB) được lấy từ Trace Cache. Các BB là các đoạn mã liên tục trong chương trình mà không có câu lệnh nhảy.
- From Branch Predictor: Trace Cache sử dụng dự đoán điều kiện nhảy để xác định liệu có nên thêm các câu lệnh nhảy vào trace hay không.
- Next Fetch Address: Trace Cache trả về địa chỉ bắt đầu của trace tiếp theo trong bộ nhớ lệnh.
- From Instruction Latch: Trace Cache lấy các tín hiệu điều khiển từ các lệnh trong trace để điều khiển thực thi chương trình.

Tóm lại, Trace Cache giúp cải thiện hiệu suất của máy tính bằng cách lưu trữ các trace được phân tích trước đó trong bộ nhớ đệm. Khi chương trình được thực thi, Trace Cache có thể

sử dụng các trace đã được lưu trữ để tăng tốc độ thực thi chương trình.

NỘI DUNG 4:

Cơ chế của Trace Cache (Fetch Mechanism) là một kỹ thuật tối ưu hóa bộ nhớ cache trên bộ xử lý để cải thiện hiệu suất của hệ thống. Đây là một kỹ thuật đặc biệt được sử dụng trong bộ xử lý của các máy tính có kiến trúc x86.

Cơ chế tìm nạp (Fetch Mechanism) của Trace Cache hoạt động bằng cách lưu trữ các đoạn mã đã được thực thi trước đó và sử dụng chúng để tìm nạp các đoạn mã tương tự trong tương lai. Khi một instruction được thực thi, Trace Cache sẽ lưu trữ các instruction kế tiếp trong đoạn mã tương ứng (instruction trace) vào bộ đệm của nó. Các đoạn mã này được gọi là "traces". Khi bộ xử lý cần thực thi một đoạn mã mới, Trace Cache sẽ truy xuất bộ đệm của nó để tìm các instruction tương tự đã được lưu trữ trước đó và đưa chúng vào bộ nhớ đệm để thực thi.

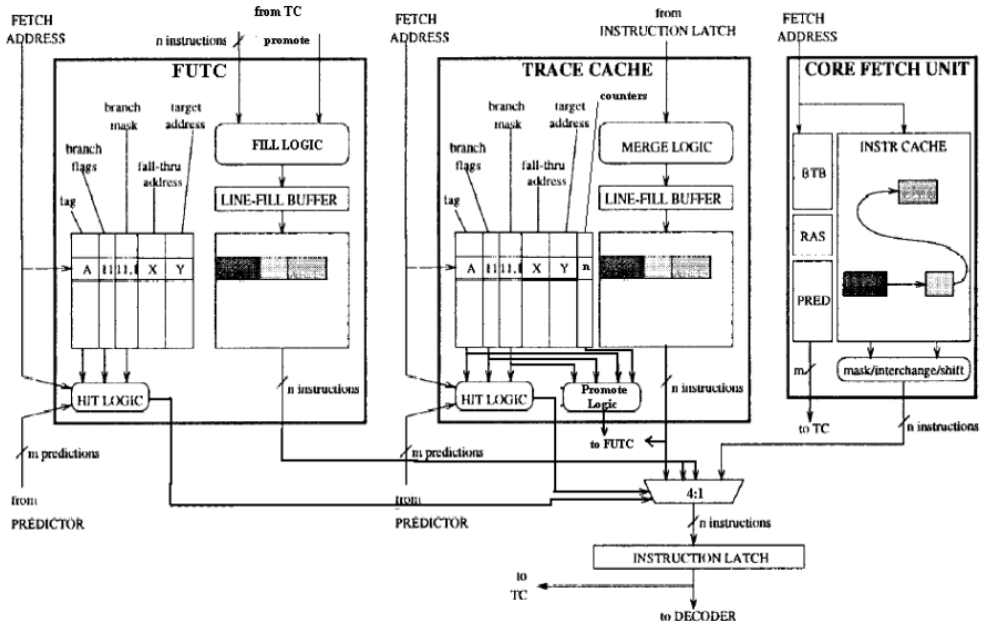
Cơ chế tìm nạp này mang lại nhiều lợi ích cho hiệu suất của bộ vi xử lý.

- Thứ nhất, Trace Cache giảm số lần truy cập bộ nhớ để lấy các instruction, giảm thời gian truy xuất.
- Thứ hai, Trace Cache tăng khả năng dự đoán nhánh (branch prediction) bằng cách lưu trữ các instruction đã được thực thi trước đó trong các đoạn mã có cấu trúc tương tự. Điều này giúp bộ vi xử lý có thể dự đoán các nhánh (branch) của các đoạn mã tương tự trong tương lai một cách chính xác hơn, giảm số lần nhảy nhánh (branch misprediction) và tăng tốc độ thực thi.

Tuy nhiên, cơ chế tìm nạp của Trace Cache cũng có một số hạn chế.

- Thứ nhất, nó chỉ hoạt động tốt khi các đoạn mã có cấu trúc tương tự và có thể lưu trữ trong bộ đệm của nó. Các đoạn mã có cấu trúc phức tạp hơn hoặc không lặp lại nhiều lần có thể không được tối ưu hóa bởi Trace Cache.

- Thứ hai, kích thước bộ đệm của Trace Cache có giới hạn, điều này có thể dẫn đến các instruction không thể lưu trữ và phải được lấy từ bộ nhớ bên ngoài, làm giảm hiệu suất của Trace Cache.



(Figure 3: Fetch Pipeline augmented with FUTC; Figure modified from [1])

NỘI DUNG 5:

1. Cache Hit là gì?

Một cache hit là trạng thái trong đó: dữ liệu được yêu cầu, được xử lý bởi một thành phần hoặc một ứng dụng được tìm thấy trong bộ nhớ cache. Đây là hình thức cung cấp dữ liệu với tốc độ nhanh chóng hơn cho bộ xử lý vì bộ nhớ cache lúc này đã chứa sẵn dữ liệu được yêu cầu đó rồi.

Cache hit xảy ra khi nội dung mà client yêu cầu được tìm thấy trong bộ nhớ cục bộ (local storage) là cập nhật nhất khi so sánh với cùng một nội dung trên máy chủ gốc (origin server).

Cụ thể quy trình của Cache hit như sau:

Một cache hit xảy ra khi một ứng dụng hoặc phần mềm có request về dữ liệu. Đầu tiên, bộ xử lý trung tâm (CPU) tìm kiếm dữ liệu ở vị trí bộ nhớ gần nhất, thường là primary cache (bộ nhớ đệm chính). Nếu dữ liệu được yêu cầu được tìm thấy trong bộ nhớ cache, nó được gọi là cache hit.

Một cache hit sẽ luôn có khả năng phục vụ dữ liệu nhanh hơn, vì lúc này dữ liệu có thể được truy xuất bằng cách đọc bộ nhớ cache. Cache hit cũng có thể nằm trong những dish cache, đây là nơi lưu trữ các dữ liệu được yêu và truy cập lần đầu tiên.

Có thể thấy cache hit sẽ gắn liền với những sự xuất hiện lần đầu của dữ liệu.

2. Cache Miss là gì?

Một cache miss là trạng thái nơi dữ liệu được yêu cầu xử lý bởi một thành phần hoặc ứng dụng không được tìm thấy trong bộ nhớ cache (cache memory). Nó gây ra sự chậm trễ trong các thực thi bởi chúng cần yêu cầu các chương trình hoặc ứng dụng tìm nạp dữ liệu từ các cấp bộ nhớ cache khác hoặc bộ nhớ chính. Vì không có sẵn các dữ liệu được yêu cầu

Trái ngược với cache hit, Cache miss xảy ra khi nội dung mà client yêu cầu hoặc không được lưu trong bộ nhớ cache, hoặc có được lưu trong bộ nhớ cache nhưng đã hết hạn và nội dung không được cập nhật khi so sánh với cùng một nội dung trên máy chủ gốc.

Cụ thể cơ chế của cache miss như sau:

Cache miss xảy ra trong các mô hình và phương thức truy cập bộ cache memory. Đối với mỗi yêu cầu mới, bộ xử lý sẽ ưu tiên tìm kiếm dữ liệu đó trong primary cache trước tiên. Nếu dữ liệu không được tìm thấy tại đây, nó được coi là một cache miss.

Mỗi cache miss sẽ góp phần làm chậm quá trình xử lý tổng thể bởi vì sau một cache miss, bộ xử lý trung tâm (CPU) sẽ tìm một bộ nhớ đệm có mức cao hơn, chẳng hạn như L1, L2, L3 và bộ nhớ truy cập ngẫu nhiên (RAM) cho dữ liệu đó. Hơn nữa, một entry mới còn phải được tạo ra và được sao chép trong cache trước khi nó có thể được truy cập bởi bộ vi xử lý.

3. Cache hit và Cache miss khác nhau như thế nào?

Sự khác biệt giữa Cache hit và Cache miss

Một cache hit nghĩa là yêu cầu của người dùng đối với một nội dung được thỏa mãn bởi phần tử của bộ nhớ cache (cache máy tính/ web cache/ CDN edge server,...), tức là nội dung được yêu cầu được trình bày chính xác trong phần tử đó của cache. Kết quả là, phần tử trong bộ nhớ cache gửi lại dữ liệu nội dung cho người dùng cuối khiến sự phản hồi trở nên nhanh chóng hơn.

Bên cạnh đó, cache miss liên quan đến một máy chủ cache cụ thể có nghĩa là nội dung được yêu cầu không có sẵn trong bộ nhớ, do đó, request phải được chuyển tiếp đến các cache ở cấp cao hơn để thực hiện truy vấn. Trong trường hợp xấu nhất, request của người dùng sẽ phải gửi đến tận máy chủ lưu trữ gốc (chẳng hạn như server HTTP). Do đó, việc phân phối nội dung tương ứng trở lại người dùng có thể bị trễ (thường là lâu hơn). Điều này có thể dẫn đến đóng băng chuyển động (rebuffering) trong các ứng dụng phát trực tuyến video.

4. Mối liên hệ của cache hit và miss logic

Cache hit và cache miss logic là hai khái niệm liên quan đến việc quản lý và sử dụng cache trong hệ thống máy tính. Khi CPU cần truy cập dữ liệu hoặc hướng dẫn, nó trước tiên kiểm tra xem nó đã được lưu trữ trong cache hay chưa.

Nếu dữ liệu hoặc hướng dẫn đã được lưu trữ trong cache, CPU có thể truy cập nhanh chóng và trực tiếp từ cache mà không cần phải truy cập đến bộ nhớ chính. Điều này được gọi là cache hit.

Ngược lại, nếu dữ liệu hoặc hướng dẫn chưa được lưu trữ trong cache, CPU phải truy cập đến bộ nhớ chính để lấy nó. Điều này được gọi là cache miss.

Do đó, có mối liên hệ trực tiếp giữa cache hit và cache miss. Khi tỉ lệ cache hit càng cao, nghĩa là cache đã lưu trữ nhiều dữ liệu hoặc hướng dẫn được sử dụng thường xuyên, CPU có thể truy cập nhanh chóng và giảm thời gian đợi truy cập bộ nhớ chính, giúp cải thiện hiệu suất của hệ thống.

Ngược lại, khi tỉ lệ cache miss càng cao, CPU phải truy cập đến bộ nhớ chính nhiều hơn, làm tăng thời gian đợi và giảm hiệu suất của hệ thống.

Vì vậy, để tăng hiệu suất của hệ thống, ta cần tối ưu hóa việc sử dụng cache để giảm tỉ lệ cache miss và tăng tỉ lệ cache hit. Điều này có thể được thực hiện bằng cách sử dụng các chiến lược quản lý cache thông minh và tối ưu hóa kích thước và cấu trúc của cache.

5. Tác dụng của cache hit và miss logic

Tác dụng của cache hit và miss logic là cải thiện hiệu suất của hệ thống máy tính. Khi CPU cần truy cập dữ liệu hoặc hướng dẫn, nó trước tiên kiểm tra xem chúng đã được lưu trữ trong cache hay chưa.

Nếu dữ liệu hoặc hướng dẫn đã được lưu trữ trong cache, CPU có thể truy cập nhanh chóng và trực tiếp từ cache mà không cần phải truy cập đến bộ nhớ chính. Điều này được gọi là cache hit. Việc cache hit giúp cải thiện hiệu suất của hệ thống bởi vì nó giảm thời gian truy cập đến bộ nhớ chính và tăng tốc độ xử lý của CPU.

Ngược lại, nếu dữ liệu hoặc hướng dẫn chưa được lưu trữ trong cache, CPU phải truy cập đến bộ nhớ chính để lấy chúng. Điều này được gọi là cache miss. Việc cache miss sẽ làm giảm hiệu suất của hệ thống bởi vì nó tạo ra độ trễ trong việc truy cập dữ liệu hoặc hướng dẫn.

Do đó, để cải thiện hiệu suất của hệ thống, ta cần tối ưu hóa việc sử dụng cache để giảm tỉ lệ cache miss và tăng tỉ lệ cache hit. Các chiến lược quản lý cache thông minh và tối ưu hóa kích thước và cấu trúc của cache là những cách để đạt được mục tiêu này.

❖Trần Duy Khánh

NỘI DUNG 3:

*Thiết kế của Trace Cache bao gồm một bộ đệm lịch sử các lệnh mà CPU đang thực thi. Khi CPU gặp phải một lệnh rẻ nhánh (branch instruction), Trace Cache sẽ tra cứu bộ đệm lịch sử để tìm kiếm lựa chọn hướng đi mà CPU đã chọn trong quá khứ cho lệnh tương tự. Nếu lựa chọn này khớp với hướng đi của lệnh rẻ nhánh hiện tại, CPU có thể trực tiếp thực thi lệnh được lưu trong Trace Cache, giúp CPU hoạt động nhanh hơn so với trường hợp phải đợi lựa chọn hướng đi từ bộ nhớ bên ngoài.

*Thiết kế của Trace Cache được tối ưu hóa để đảm bảo tính khả dụng và đáng tin cậy của dữ liệu lịch sử được lưu trữ. Với các kỹ thuật như việc theo dõi các hướng đi của lệnh, việc sử dụng các kỹ thuật nhận diện mẫu và các bộ nhớ cache phụ trợ, Trace Cache có thể đưa ra các lựa chọn hướng đi chính xác hơn và giúp CPU hoạt động hiệu quả hơn.

→Tóm lại, Trace Cache là một thành phần quan trọng trong kiến trúc bộ vi xử lý (CPU) hiện đại. Thiết kế của Trace Cache bao gồm một bộ đệm lịch sử các lệnh mà CPU đang thực thi, giúp tăng tốc độ và độ chính xác của CPU. Nó cũng được tối ưu hóa để đảm bảo tính khả dụng và đáng tin cậy của dữ liệu lịch sử được lưu trữ thông qua việc sử dụng các kỹ thuật nhận diện mẫu và các bộ nhớ cache phụ trợ.

NỘI DUNG 4:

Trace Cache là một bộ nhớ cache đặc biệt được sử dụng trong các vi xử lý có kiến trúc Superscalar hoặc VLIW để tăng hiệu suất tìm nạp lệnh. Trace Cache lưu trữ các chuỗi liên tục của các lệnh đã được thực thi gần đây nhất và cung cấp chúng cho bộ phận tìm nạp (fetch unit) thay vì yêu cầu bộ phận này phải tìm nạp từ bộ nhớ chính.

Cơ chế tìm nạp (fetch mechanism) của Trace Cache hoạt động như sau:

1. Bộ phận tìm nạp (fetch unit) sẽ tìm kiếm trong Trace Cache để xem liệu có một chuỗi liên tục của các lệnh đã được lưu trữ trong cache hay không.

2. Nếu có, bộ phận tìm nạp sẽ lấy chuỗi lệnh đó từ Trace Cache và cung cấp cho bộ giải mã (decode unit) để giải mã các lệnh.

3. Nếu không có chuỗi lệnh nào được tìm thấy trong Trace Cache, bộ phận tìm nạp sẽ yêu cầu bộ nhớ chính cung cấp các lệnh mới nhất để thực thi.

Cơ chế tìm nạp của Trace Cache có nhiều lợi ích. Đầu tiên, nó giảm thiểu số lượng truy cập vào bộ nhớ chính, giúp giảm độ trễ và tăng tốc độ thực thi lệnh. Thứ hai, Trace Cache cũng giúp tăng hiệu suất của các vi xử lý Superscalar hoặc VLIW bằng cách cung cấp các chuỗi lệnh liên tục để thực thi đồng thời, thay vì chỉ thực thi từng lệnh một.

Tuy nhiên, Trace Cache cũng có một số hạn chế:

- Đầu tiên, nó yêu cầu một lượng bộ nhớ đáng kể để lưu trữ các chuỗi lệnh, do đó có thể ảnh hưởng đến kích thước của bộ nhớ cache và tăng chi phí sản xuất.
- Thứ hai, Trace Cache có thể không hiệu quả khi các chuỗi lệnh không liên tục hoặc khi các lệnh được thực thi theo một cách không đồng nhất.

NỘI DUNG 5:

1. Giới thiệu:

Trong kiến trúc CPU hiện đại, Trace Cache là một trong những thành phần quan trọng giúp tăng tốc độ thực thi các chương trình máy tính. Tuy nhiên, việc xử lý Trace Cache Miss và Hit Logic có ảnh hưởng lớn đến hiệu suất của hệ thống. Trong báo cáo này, chúng ta sẽ tìm hiểu về Trace Cache, Trace Cache Miss và Hit Logic cùng với những cách giải quyết để tối ưu hoá hiệu quả của Trace Cache.

2. Trace Cache Miss và Hit Logic:

Khi CPU yêu cầu truy cập đến lệnh mà chưa được lưu trong Trace Cache, Trace Cache Miss sẽ xảy ra. Khi đó, CPU sẽ phải tìm kiếm các lệnh trong bộ nhớ RAM hoặc những bộ nhớ cache khác, dẫn đến tốn thời gian và giảm hiệu suất.

Trong khi đó, Trace Cache Hit sẽ xảy ra khi CPU tìm thấy các lệnh trong Trace Cache mà nó cần, giúp tăng tốc độ xử lý chương trình.

Chính vì vậy, việc giảm lỗi Trace Cache Miss và tối ưu hoá Hit Logic là rất quan trọng để tăng cường hiệu suất của CPU.

3. Cách giải quyết:

Các nhà sản xuất CPU đã nghiên cứu và phát triển nhiều giải pháp để giảm thiểu lỗi Trace Cache Miss và tối ưu hoá Trace Cache Hit Logic. Một số giải pháp như sau:

- Tăng kích thước của Trace Cache để cải thiện khả năng lưu trữ các trace của chương trình.

- Sử dụng các kỹ thuật dự đoán nhánh để giảm thiểu Trace Cache Miss.

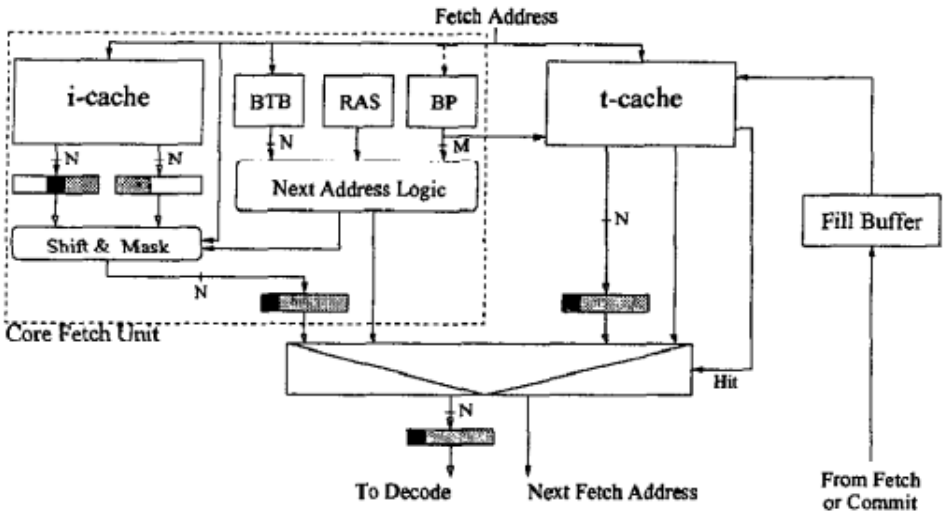
- Tối ưu hoá cơ chế kiểm tra dữ liệu tại Trace Cache Hit để giảm thiểu lỗi cache và tăng tốc độ xử lý.

4. Kết luận:

Trace Cache Miss và Hit Logic là hai vấn đề quan trọng trong kiến trúc CPU hiện đại. Việc giảm thiểu lỗi Trace Cache Miss và tối ưu hoá Hit Logic sẽ giúp tăng cường hiệu suất của CPU và cải thiện khả năng xử lý chương trình. Các nhà sản xuất CPU đã nghiên cứu và phát triển nhiều giải pháp để giải quyết vấn đề này, và chúng ta cần phải tiếp tục theo dõi và tìm hiểu để nâng cao hiệu quả sử dụng CPU.

❖Nguyễn Minh Hùng

NỘI DUNG 4:



Mô phỏng một đơn vị tìm nạp tuần tự tích cực như được hiển thị ở hình trên, mô hình đơn vị tìm nạp cốt lõi này bao gồm một bộ đệm chỉ dẫn xen kẽ (i-cache), một bộ dự báo nhiều nhánh (BP), một bộ đệm đích nhánh xen kẽ (BTB) và một ngăn xếp địa chỉ trả về (RAS). Hai dòng i-cache liên tiếp được truy cập trong mỗi chu kỳ, cho phép tìm nạp mã tuần tự vượt qua ranh giới dòng bộ đệm. BTB được truy cập song song với i-cache và được sử dụng để dự đoán địa chỉ của các bước nhảy gián tiếp và lệnh gọi chương trình con. Địa chỉ trả về của các chương trình con có thể được dự đoán chính xác bằng RAS. Mục tiêu của các nhánh liên quan đến PC được tính toán, không thu được từ BTB. Sử dụng kết quả đầu ra từ BP, BTB và thông tin liên quan đến chỉ dẫn nào đại diện cho việc truyền điều khiển, chúng ta có được chỉ dẫn ẩn để chọn chỉ dẫn hợp lệ từ các dòng i-cache đã tìm nạp và tạo địa chỉ tìm nạp cho chu kỳ tiếp theo. Thêm một bit ngắt vào BTB, bit này sẽ thông báo cho đơn vị tìm nạp khi địa chỉ bước nhảy được dự đoán sẽ phá vỡ chuỗi.

Tóm lại, quá trình tìm nạp lệnh dừng ở một trong các điều kiện sau:

- 16 chỉ dẫn được tìm nạp
- 3 chi nhánh được lấy
- Một chi nhánh được dự đoán lấy

- Một bước nhảy gián tiếp được dự đoán sẽ phá vỡ trình tự thực hiện
- Một cuộc gọi hệ thống được tìm nạp
- Trên bất kỳ dự đoán sai hoặc BTB bỏ lỡ

❖Lê Trọng Tài

NỘI DUNG 3:

Sau đây là các yếu tố cần được xem xét trong khi thiết kế bộ đệm theo dõi.

- Chính sách lựa chọn theo dõi - *số lượng lệnh tối đa* và *khối cơ bản tối đa trong một lần theo dõi*
- Tính kết hợp - số cách bộ đệm có thể có
- Phương pháp lập chỉ mục bộ đệm - nối hoặc XOR với các bit PC
- Tính kết hợp đường dẫn - các dấu vết với cùng một PC khởi động nhưng với các khối cơ bản khác nhau có thể được ánh xạ tới các bộ khác nhau
- Theo dõi các lựa chọn điền vào bộ đệm -
    1. Sau giai đoạn giải mã (suy đoán)
    2. Sau giai đoạn nghỉ hưu

Bộ đệm theo dõi không nằm trên đường tìm nạp lệnh quan trọng

NỘI DUNG 4:

Chu trình tìm nạp giải mã thực thi, hoặc đơn giản là chu trình tìm nạp-thực thi là chu trình mà bộ xử lý trung tâm (CPU) tuân theo từ khi khởi động cho đến khi máy tính tắt để xử lý các lệnh. Nó bao gồm ba giai đoạn chính: giai đoạn tìm nạp, giai đoạn giải mã và giai đoạn thực thi.

Trong các CPU đơn giản hơn, chu kỳ lệnh được thực hiện tuần tự, mỗi lệnh được xử lý trước khi lệnh tiếp theo được bắt đầu. Trong hầu hết các CPU hiện đại, thay vào đó, các chu kỳ lệnh được thực thi đồng thời và thường là song song , thông qua một đường dẫn lệnh : lệnh tiếp theo bắt đầu được xử lý trước khi lệnh trước đó kết thúc, điều này có thể xảy ra do chu kỳ được chia thành các bước riêng biệt.

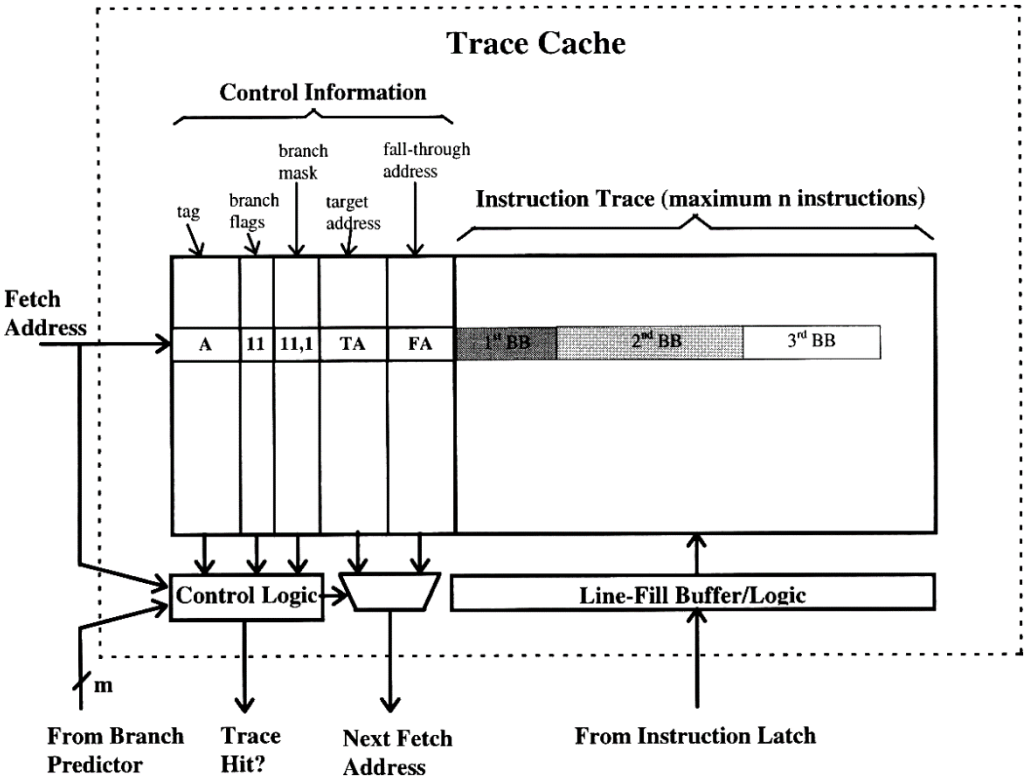
NỘI DUNG 5:

Các đường theo dõi được lưu trữ trong bộ đệm theo dõi dựa trên PC của lệnh đầu tiên trong theo dõi và một tập hợp các dự đoán rẽ nhánh. Điều này cho phép lưu trữ các đường theo dõi khác nhau bắt đầu trên cùng một địa chỉ, mỗi đường đại diện cho các kết quả nhánh khác nhau. Phương pháp gắn thẻ này giúp cung cấp khả năng kết hợp đường dẫn đến bộ nhớ cache theo dõi. Phương pháp khác có thể bao gồm việc chỉ khởi động PC dưới dạng thẻ trong bộ đệm theo dõi. Trong giai đoạn tìm nạp lệnh của một đường ống , PC hiện tại cùng với một tập hợp các dự đoán nhánh được kiểm tra trong bộ nhớ cache theo dõi để tìm lần truy cập . Nếu có một lần truy cập, một dòng theo dõi sẽ được cung cấp cho đơn vị tìm nạp mà không cần phải chuyển đến bộ đệm thông thường hoặc bộ nhớ cho các hướng dẫn này. Bộ đệm theo dõi tiếp tục cung cấp cho đơn vị tìm nạp cho đến khi dòng theo dõi kết thúc hoặc cho đến khi có một dự đoán sai trong đường ống. Nếu có sai sót, một dấu vết mới sẽ bắt đầu được xây dựng.

Bộ đệm theo dõi thực thi của Pentium 4 lưu trữ các hoạt động vi mô do giải mã các lệnh x86 , cung cấp chức năng của bộ đệm hoạt động vi mô. Có được điều này, lần sau khi cần một lệnh, lệnh đó không cần phải được giải mã thành các hoạt động vi mô nữa.

❖ NỘI DUNG SAU KHI ĐÃ ĐƯỢC ĐÚC KẾT LẠI

NỘI DUNG 3:



Hình 1. SƠ ĐỒ CẤU TẠO CỦA CACHE

Đây một sơ đồ khối của Trace Cache, một cấu trúc bộ nhớ đệm đặc biệt được sử dụng trong kiến trúc máy tính để cải thiện hiệu suất trong việc phân tích, dự đoán và thực thi các lệnh.

Cấu trúc của 1 trace cache bao gồm 2 phần: thông tin điều khiển (Control Information) và Vết chỉ dẫn (Instruction Trace).

Phần Control Information lại bao gồm Control Logic và địa chỉ của ô nhớ sẽ được tìm nạp tiếp theo hay còn gọi là Next Fetch Address.

Trong phần Control Logic, Trace Cache sẽ nhận địa chỉ tìm nạp và thông qua tag, branch flags, branch mask để biết được trace hit hay là trace miss. Nếu là trace miss thì trace cache này phải đợi. Còn là trace hit thì sẽ chuyển đến Fetch Address tiếp theo.

Điểm qua về khái niệm của 1 số thuật ngữ sử dụng trong Trace Cache:

- Tag ở đây có tác dụng xác định địa chỉ bắt đầu của vết, tức là các bit bậc cao hơn của địa chỉ cho chỉ dẫn đầu tiên của vết.
- Branch Flags là một bit duy nhất cho mỗi nhánh trong vết để cho biết có hay không nhánh đã được thực hiện hoặc không được thực hiện.
- Branch Mask: Cần có một số trạng thái để chỉ ra số lượng nhánh trong dấu vết và điều này cần Branch Mask quyết định. Thông tin này được cả bộ đệm theo dõi sử dụng để xác định có bao nhiêu cờ nhánh cần kiểm tra và bởi bộ dự đoán nhánh để cập nhật.
- Đến với phần Fetch Address tiếp theo thì ta có Target Address và Fall-through Address.
- Target Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng là một nhánh và đượxc dự đoán là đã thực hiện.
- Fall-through Address sẽ là địa chỉ tìm nạp tiếp theo nếu lệnh cuối cùng của vết là một nhánh và được dự đoán là không được thực hiện.

Instruction cache được tạo ra bởi Line-Fill Buffer / Logic hay còn gọi là bộ đệm điền dòng hoặc logic điền dòng.

Bộ đệm điền dòng được sử dụng để chốt các khối cơ bản lại với nhau để tạo thành các vết được ghi vào trace cache. Các khối cơ bản được chốt từng cái một vào bộ đệm điền khi tiến hành tìm nạp lệnh. Logic điền (Fill logic) sẽ hợp nhất từng khối cơ bản mới với các hướng dẫn đã có trong bộ đệm điền vào dòng. Quá trình điền hoàn tất khi đạt được độ dài vết tối đa n hoặc số lượng nhánh tối đa trong vết m.